Intro to Deep Neural Networks

CSCI-P 556

ZORAN TIGANJ

Today: Intro to Deep Neural Networks

- Intro to Deep Neural Networks (Chapter 10)
  - Artificial Neural Network (ANN): a network composed of artificial neurons
  - Deep Neural Network (DNN): an ANN with multiple layers
  - Multi-layer Perceptron (MLP): DNN with multiple fully connected and stacked layers (simplest form of DNN)

Brief history of ANNs

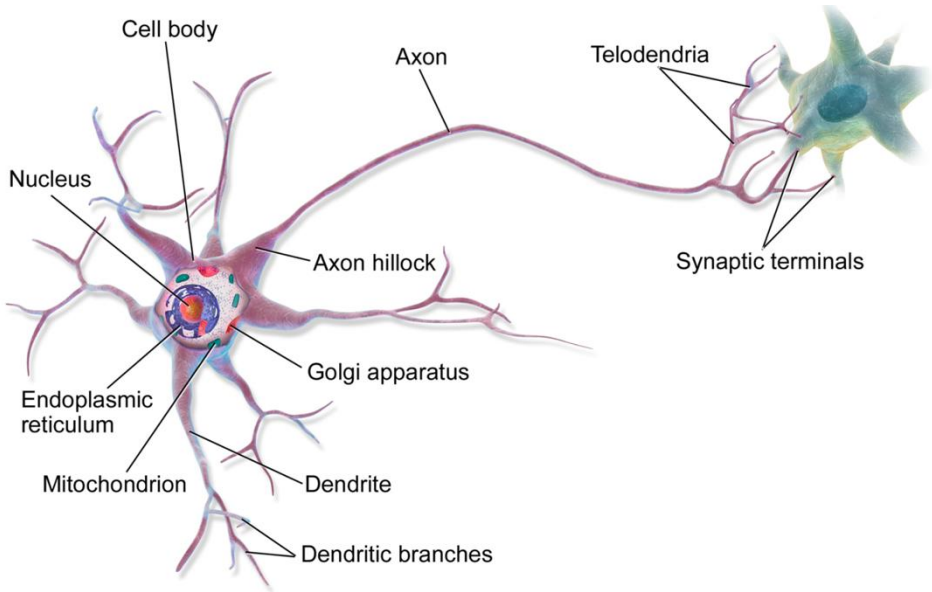
- ANNs were first introduced back in 1943 by the neurophysiologist Warren McCulloch and the mathematician Walter Pitts.
- The early successes of ANNs led to the widespread belief that we would soon be conversing with truly intelligent machines. When it became clear in the 1960s that this promise would go unfulfilled (at least for quite a while), funding flew elsewhere (first AI winter)
- In the early 1980s, new architectures were invented and better training techniques were developed, sparking a revival of interest in connectionism (the study of neural networks).
- But progress was slow, and by the 1990s other powerful Machine Learning techniques were invented, such as SVM (second AI winter)

Brief history of ANNs

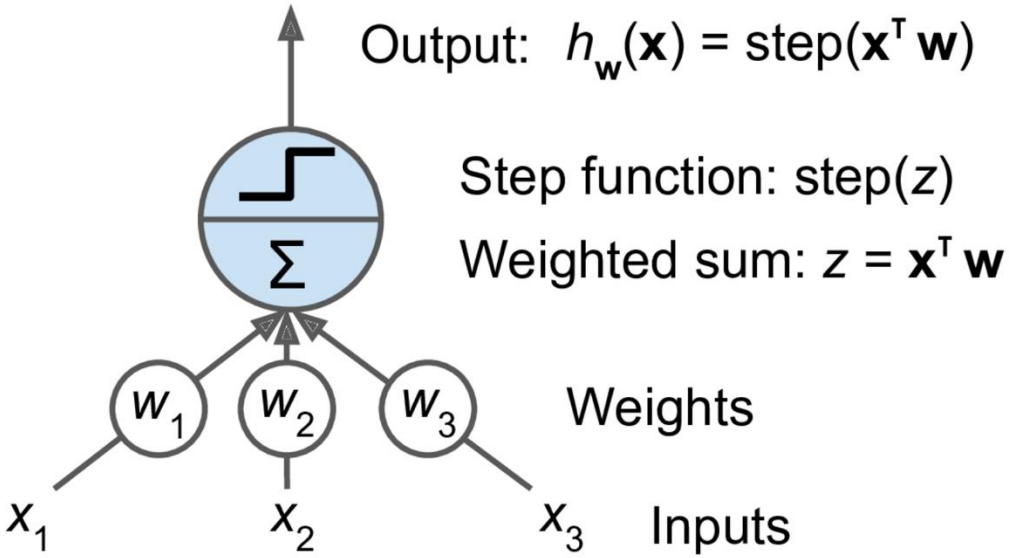
- We are now witnessing yet another wave of interest in ANNs.
- What changed?
  - There is now a huge quantity of data available to train neural networks.
  - The tremendous increase in computing power since the 1990s.
  - The training algorithms have been improved (slightly).
  - Some theoretical limitations of ANNs have turned out to be benign in practice (like getting stuck in local minimum).
  - ANNs seem to have entered a virtuous circle of funding and progress.

Biological and artificial neurons

Biological neuron

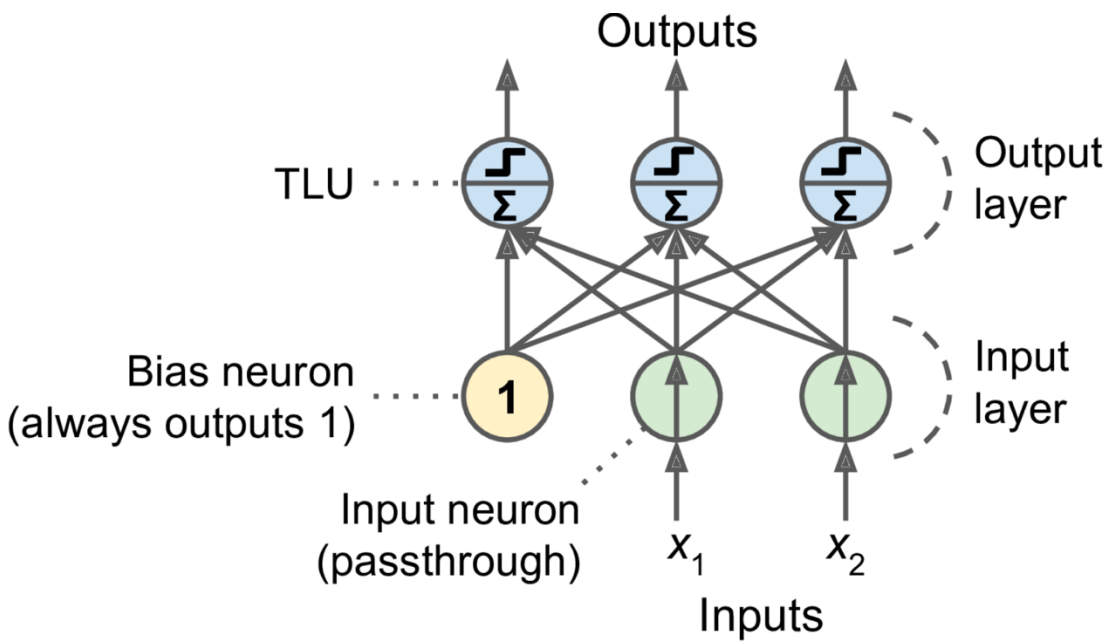


Artificial neuron (perceptron)



Modern artificial neurons will have a differentiable activation function (e.g. sigmoid) instead of the step function

Combining neurons into networks

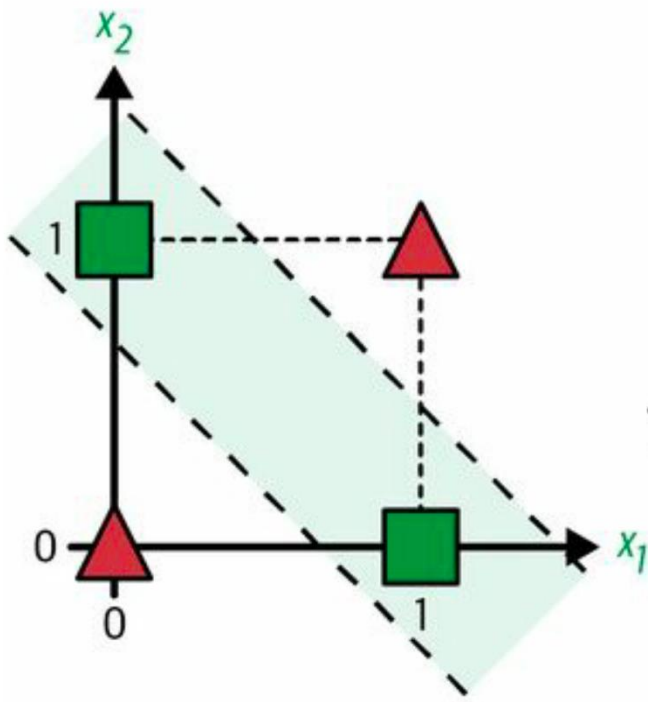


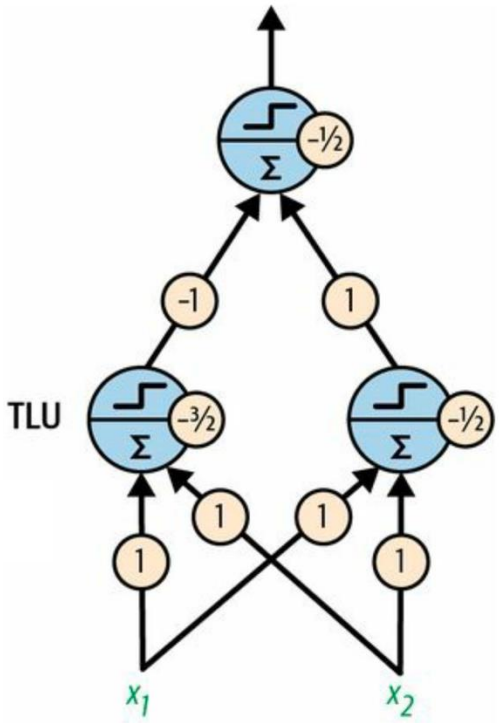
$$h_{\mathbf{W},\mathbf{b}}(\mathbf{X}) = \phi(\mathbf{X}\mathbf{W} + \mathbf{b})$$

- As always, **X** represents the matrix of input features. It has one row per instance and one column per feature.
- **The weight matrix W** contains all the connection weights except for the ones from the bias neuron. It has one row per input neuron and one column per artificial neuron in the layer.
- **The bias vector b** contains all the connection weights between the bias neuron and the artificial neurons. It has one bias term per artificial neuron.
- $\boldsymbol{\phi}$ is the activation function

XOR problem and perceptron

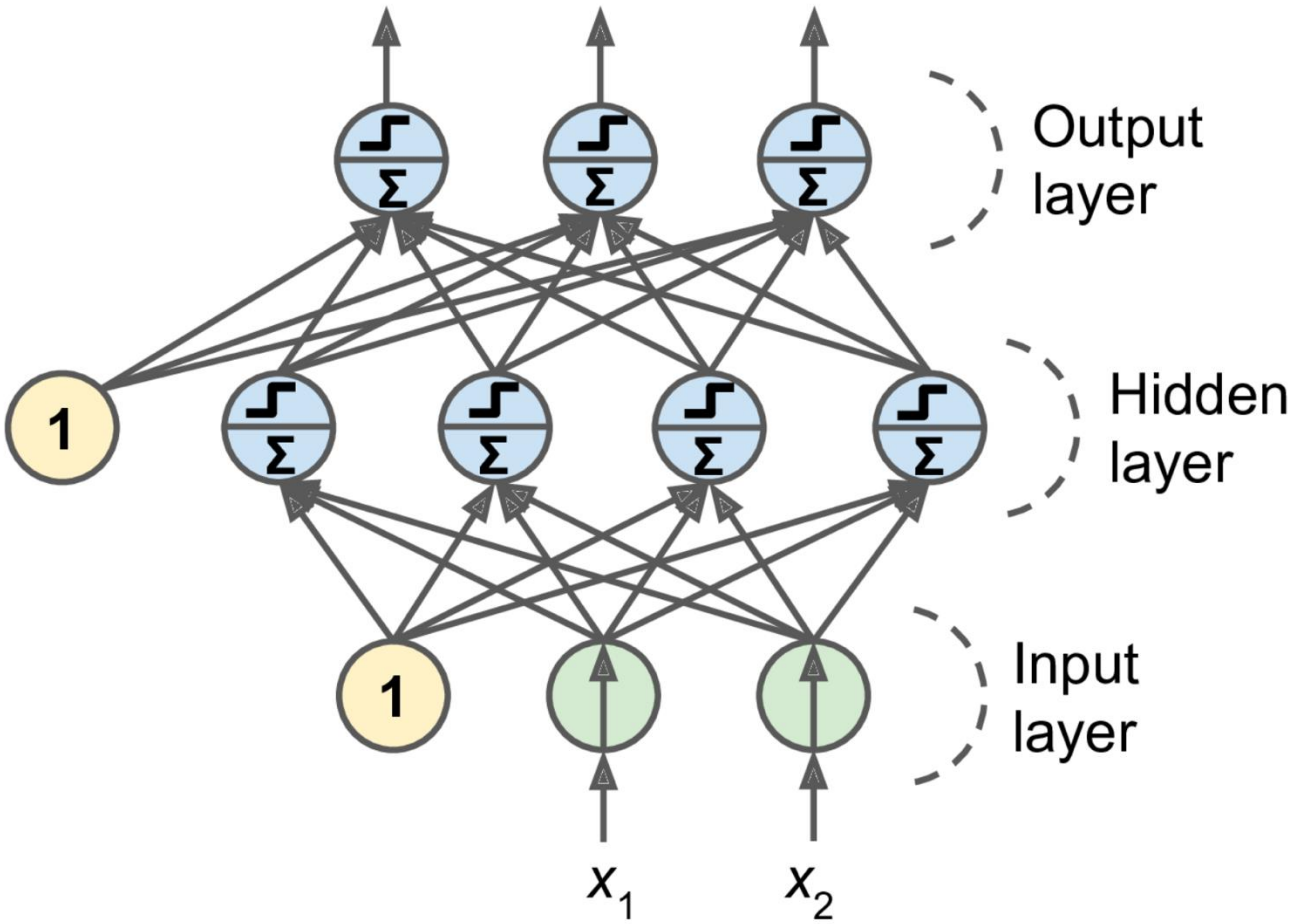
Can we solve XOR problem with a single neuron?





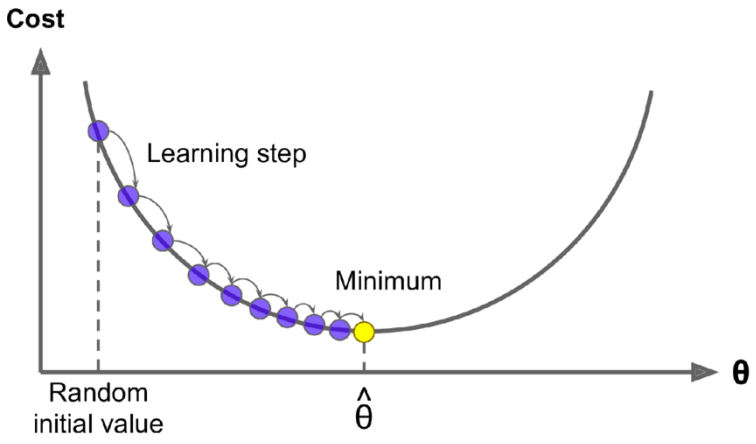
The perceptron is one of the simplest ANN architectures, invented in 1957 by Frank Rosenblatt. It is based on a threshold logic unit (TLU),

Deep Neural Networks



How to train DNNs

- In1986, David Rumelhart, Geoffrey Hinton, and Ronald Williams published a groundbreaking paper that introduced the backpropagation training algorithm, which is still used today.
- Backpropagation is based on Gradient Descent: the backpropagation algorithm is able to compute the gradient of the network's error with regard to every single model parameter.
- In other words, it can find out how each connection weight and each bias term should be tweaked in order to reduce the error.
- Once it has these gradients, it just performs a regular Gradient Descent step, and the whole process is repeated until the network converges to the solution.

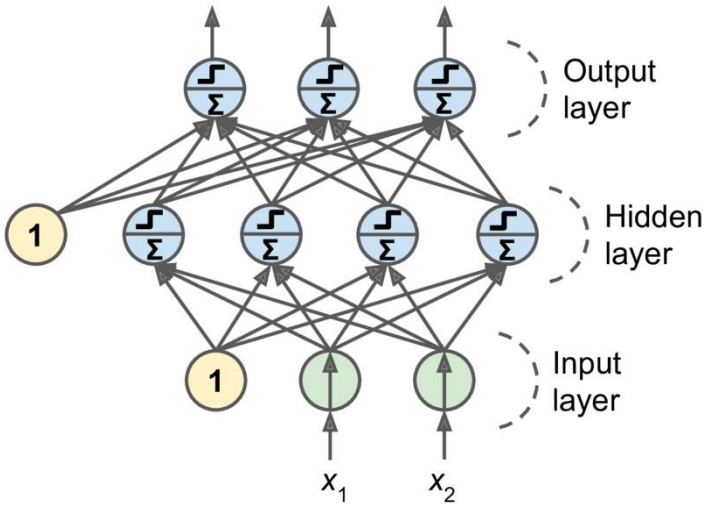


$$\boldsymbol{\theta}^{(\text{next step})} = \boldsymbol{\theta} - \eta \nabla_{\boldsymbol{\theta}} \operatorname{MSE}(\boldsymbol{\theta})$$

More about backpropagation algorithm

- For each training instance:
  - the backpropagation algorithm first makes a prediction (forward pass) and measures the error,
  - then goes through each layer in reverse to measure the error contribution from each connection (reverse or backward pass),
  - and finally tweaks the connection weights to reduce the error (Gradient Descent step).

$$\boldsymbol{\theta}^{(\text{next step})} = \boldsymbol{\theta} - \eta \nabla_{\boldsymbol{\theta}} \operatorname{MSE}(\boldsymbol{\theta})$$

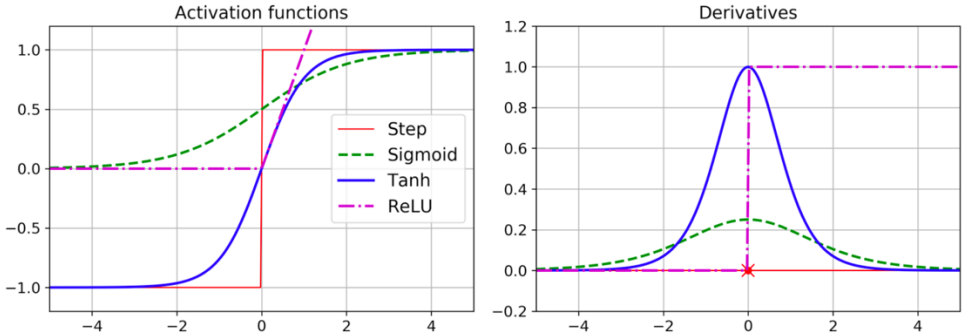


Weight initialization

- What should be the initial weights?
- It is important to initialize all the hidden layers' connection weights randomly, or else training will fail.
- Why?
  - For example, if you initialize all weights and biases to zero, then all neurons in a given layer will be perfectly identical, and thus backpropagation will affect them in exactly the same way, so they will remain identical. In other words, despite having hundreds of neurons per layer, your model will act as if it had only one neuron per layer.

Activation functions

- Activation function has to be differentiable so we can compute the gradient.
- Some examples of activations functions:
  - Sigmoid $\sigma(z) = 1 / (1 + \exp(-z))$
  - Hyperbolic tangent function $\tanh(z) = 2\sigma(2z) - 1$
    - To make each layer's output more or less centered around 0 at the beginning of training, which often helps speed up convergence.
  - Rectified Linear Unit function: $\mathrm{ReLU}(z) = \max(0, z)$
    - The fact that it does not have a maximum output value helps reduce some issues during Gradient Descent.



Why do we need nonlinearity?

- Why do we need nonlinearity in activation functions?
- If you chain several linear transformations, all you get is a linear transformation.
  - For example, if $f(x) = 2x + 3$ and $g(x) = 5x - 1$, then chaining these two linear functions gives you another linear function: $f(g(x)) = 2(5x - 1) + 3 = 10x + 1$.
  - So if you don't have some nonlinearity between layers, then even a deep stack of layers is equivalent to a single layer, and you can't solve very complex problems with that.
  - Conversely, a large enough DNN with nonlinear activations can theoretically approximate any continuous function.

Next time

- Training with Intro to Deep Neural Networks: Building simple DNNs (chapter 10)